

“AI调试对决”实验报告

Portra约拍服务平台

Phase 4 — AI 调试对决实验报告

项目：Portra 约拍服务平台
阶段：P4 模块集成、前后端联调与工程化收尾
实验编号：P4-EXP-AI-DEBUG-DUEL-TEAM-01
整理日期：2026-06-07

一、实验目标

P4 要求团队在系统联调阶段选择 2–3 个具有代表性的 Bug，比较纯人工调试和 AI 辅助调试的差异。

本报告选择 3 个跨模块问题：

Bug	涉及范围
Bug 1：拍摄状态人工推进与系统推进冲突	订单 Controller、Service、状态机、smoke test
Bug 2：交付上传 HTTP 回调导致数据库锁等待	Delivery、Order、数据库事务、状态回调
Bug 3：CI/CD 只覆盖后端，未覆盖前端构建和完整质量检查	GitHub Actions、GitLab CI、frontend、backend

这三个问题分别覆盖：

业务规则一致性
跨模块事务与数据库
团队工程化与质量保障

因此，本报告不是 D 线个人日志，而是团队整体联调实验。

二、实验记录口径

2.1 仓库读取范围

```
backend/src/main/java/com/action/camera/order/controller/OrderController.java
a
backend/src/main/java/com/action/camera/order/service/OrderService.java
backend/src/main/java/com/action/camera/order/statemachine/OrderStatusMachine.java
backend/src/main/java/com/action/camera/delivery/service/DeliveryService.java
a
smoke-test.spec.js
smoke_test_issues.md
.github/workflows/ci.yml
.gitlab-ci.yml
frontend/package.json
backend/pom.xml
```

2.2 时间说明

耗时数据为本阶段复盘估算值，用于比较调试路径差异。正式提交前，团队可结合终端历史、Git commit 时间和成员记录微调。

三、Bug 1：拍摄状态人工推进与系统推进冲突

3.1 问题现象

订单拍摄阶段存在互相冲突的实现：

Controller 允许手动推进

`OrderController.validateP4Transition()` 允许服务方请求：

```
PAID_PENDING_SHOOT → SHOOTING
SHOOTING → PENDING_DELIVERY
```

Service 禁止手动推进

`OrderService.ensureNotManualShootingTransition()` 明确禁止：

```
PAID_PENDING_SHOOT → SHOOTING
SHOOTING → PENDING_DELIVERY
```

并提示：

```
Shooting status is advanced by the system schedule, not manually
```

Smoke Test 仍然手动推进

`smoke-test.spec.js` Step 11 和 Step 12 调用：

```
POST /orders/{orderId}/status-transitions
targetStatus = SHOOTING
targetStatus = PENDING_DELIVERY
```

3.2 根因分析

业务规则在不同层没有同步更新：

Controller: 仍沿用早期人工推进逻辑
Service: 已经改为系统计划自动推进
Smoke Test: 仍按旧接口验证

根因不是某一行代码错误，而是**业务口径漂移**。

3.3 纯人工调试路径

人工排查步骤：

1. 运行 `smoke test`;
2. 查看 Step 11 / Step 12 失败;
3. 查看 `Controller`;
4. 查看 `Service`;
5. 查看 `OrderStatusMachine`;
6. 查找自动调度方法;
7. 判断最终产品口径;
8. 更新接口与测试。

估算耗时：

约 45 分钟

3.4 AI 辅助调试输入

`smoke-test.spec.js` 的 Step 11 和 Step 12 调用手动拍摄状态迁移。
`OrderController` 似乎允许这些迁移，但 `OrderService` 又禁止。
请对照 `Controller`、`Service`、`OrderStatusMachine` 和 `smoke test`，
指出根因，不要只绕过异常。

3.5 AI 辅助定位

AI 可以快速给出：

Controller、Service、smoke test 口径不一致
Service 中存在系统自动推进方法 autoAdvanceShootingOrders()
应统一成系统自动推进，不能只删异常

估算耗时：

约 8 分钟

3.6 最终修复方案

采用系统自动推进：

PAID_PENDING_SHOOT → SHOOTING
SHOOTING → PENDING_DELIVERY

修复步骤：

1. 从 Controller 手动暴露规则中删除这两条迁移；
2. 前端不展示“开始拍摄 / 结束拍摄”人工按钮；
3. smoke test 改为：
 - 构造合适时间；
 - 调用自动推进任务；
 - 或在测试层直接验证 autoAdvanceShootingOrders()；
4. 保留 OrderStatusMachine 合法迁移；
5. 补单元测试；
6. 更新演示说明。

3.7 验证方式

1. 手工调用状态迁移接口应被拒绝。
2. 自动任务可推进至 SHOOTING。
3. 到达 shootEndTime 后自动推进至 PENDING_DELIVERY。
4. 状态日志包含 SYSTEM 操作来源。
5. 更新后的 smoke test 通过。

3.8 结论

项目	结果
AI 定位是否准确	是
AI 是否只修表象	否
AI 方案是否可用	可用

项目	结果
最终方案来源	AI 缩小范围，团队确认系统自动推进口径
纯人工定位耗时	约 45 分钟
AI 辅助定位耗时	约 8 分钟

四、Bug 2：交付上传 HTTP 回调导致数据库锁等待

4.1 问题现象

Delivery 上传流程需要：

```
保存 deliveries
保存 delivery_files
更新 orders.status
```

订单状态更新通过 OrderStatusPort 触发本地 HTTP 回调。

如果上传方法外层直接使用一个大事务：

```
INSERT deliveries
→ 持有 deliveries → orders 外键共享锁
→ HTTP 回调另一个线程 UPDATE orders
→ 等待锁释放
→ 外层事务又等待 HTTP 返回
```

可能形成锁等待甚至死锁。

4.2 根因分析

根因是：

```
数据库事务没有在跨线程 HTTP 回调前提交
```

本质上是：

```
事务边界
外键锁
HTTP 回调
跨线程等待
```

共同导致的问题。

4.3 纯人工调试路径

人工排查步骤：

1. 复现上传交付卡住或超时；
2. 查看后端日志；
3. 检查数据库锁等待；
4. 查看 `DeliveryService` 事务；
5. 查看 `deliveries` 外键；
6. 查看 `OrderStatusPort` 回调；
7. 判断跨线程等待关系；
8. 重构事务边界；
9. 增加失败回滚。

估算耗时：

约 90 分钟

4.4 AI 辅助调试输入

上传交付时可能卡住。

`DeliveryService` 保存 `deliveries` 后，会通过本地 HTTP 回调修改 `orders`。

`deliveries` 表有 `orders.id` 外键。

请分析是否存在事务锁等待或死锁，不要只增加超时。

4.5 AI 辅助定位

AI 可以指出：

外层事务未提交

FK 共享锁仍然存在

回调线程更新 `orders` 需要排他锁

两个线程互相等待

估算耗时：

约 18 分钟

4.6 最终修复方案

真实仓库已经采用：

`upload()` 不加外层 `@Transactional`

`TransactionTemplate` 独立事务保存 `deliveries` / `delivery_files`

先提交并释放 FK 共享锁

再调用 `orderStatusPort.changeStatus()`
状态更新失败时 `rollbackSavedDelivery()`

该方案比“延长超时”或“重试 HTTP”更正确，因为它处理了根因。

4.7 验证方式

- 1. 正常上传交付成功。
- 2. 订单进入 `DELIVERED_PENDING_CONFIRM`。
- 3. `Delivery` 与 `DeliveryFile` 均存在。
- 4. 模拟状态回调失败时，交付记录被回滚删除。
- 5. 重复执行多次，不出现锁等待或超时。
- 6. 观察数据库无长期锁等待。

4.8 结论

项目	结果
AI 定位是否准确	是
AI 是否只修表象	否
AI 方案是否可用	可用
最终方案来源	AI 辅助分析锁关系，团队采用独立事务 + 失败回滚
纯人工定位耗时	约 90 分钟
AI 辅助定位耗时	约 18 分钟

五、Bug 3：CI/CD 只覆盖后端，未满足完整质量检查要求

5.1 问题现象

P4 要求 CI/CD 至少包括：

- 自动安装依赖
- 静态检查或格式检查
- 单元测试
- 集成测试
- 自动构建

仓库当前：

GitHub Actions

Backend CI

执行：

```
backend compile
backend test
backend package
```

GitLab CI

执行：

```
backend mvn test
backend mvn -DskipTests package
MySQL 服务
jar artifacts
```

但没有：

```
frontend npm install / npm ci
frontend npm run build
前端 lint 或静态检查
smoke-test.spec.js 自动执行
```

5.2 根因分析

CI 最初围绕后端稳定性搭建，前端和全链路 smoke test 后续加入后，没有同步进入流水线。

根因是：

工程化配置滞后于功能扩展

5.3 纯人工调试路径

人工排查步骤：

1. 阅读 P4 CI/CD 验收标准；
2. 查看 GitHub Actions；
3. 查看 GitLab CI；
4. 查看 frontend/package.json；
5. 查看 smoke-test.spec.js；
6. 对照缺失任务；
7. 设计新增 job。

估算耗时：

约 35 分钟

5.4 AI 辅助调试输入

请对照 P4 CI/CD 验收要求，
检查 `.github/workflows/ci.yml`、`.gitlab-ci.yml`、
`frontend/package.json`、`backend/pom.xml`、`smoke-test.spec.js`。
列出当前流水线缺口，并给出最小改动方案。

5.5 AI 辅助定位

AI 可以快速列出：

缺 `frontend job`
缺 `npm ci`
缺 `npm run build`
缺静态检查
缺自动 `smoke test`
GitHub Actions 和 GitLab CI 覆盖范围不完全一致

估算耗时：

约 7 分钟

5.6 最终修复方案

增加前端 job：

```
frontend_install  
frontend_build
```

最低要求：

```
cd frontend  
npm ci  
npm run build
```

如果补充 lint：

新增 `lint script`
在 CI 中执行 `npm run lint`

对集成测试：

```
启动 MySQL
导入 SQL
启动后端
等待 /test/success
node smoke-test.spec.js
```

同时保留：

```
backend mvn test
backend package
jar artifacts
```

5.7 验证方式

- 1. 推送 feature 分支或 merge request。
- 2. 前端依赖安装成功。
- 3. frontend npm run build 成功。
- 4. 后端测试成功。
- 5. 后端打包成功。
- 6. smoke test 自动运行。
- 7. CI 页面保留最近一次运行记录。

5.8 结论

项目	结果
AI 定位是否准确	是
AI 是否只修表象	否
AI 方案是否可用	可用
最终方案来源	AI 对照验收标准，团队决定最小流水线增量
纯人工定位耗时	约 35 分钟
AI 辅助定位耗时	约 7 分钟

六、实验记录总表

Bug #	Bug 描述	纯人工定位耗时	AI 辅助定位耗时	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
1	拍摄状态人工推进与系统自动推进冲突	约 45 分钟	约 8 分钟	是	可用	AI 定位跨层口径冲突，团队确认自动推进
2	Delivery 上传 HTTP 回调引发 FK 锁等待	约 90 分钟	约 18 分钟	是	可用	AI 辅助分析锁关系，团队采用独立事务
3	CI/CD 只覆盖后端	约 35 分钟	约 7 分钟	是	可用	AI 对照 P4 标准，团队补前端与 smoke job

平均耗时：

纯人工：约 56.7 分钟 / Bug
AI 辅助：约 11.0 分钟 / Bug
复盘估算节省：约 80.6%

七、分析结论

7.1 AI 擅长哪类 Bug

AI 表现较好的问题：

跨文件规则不一致
Controller 与 Service 契约冲突
事务边界与锁等待
CI 配置缺口
已有测试与实现不同步

原因是这些问题具有明确证据链，AI 可以快速对照多个文件。

7.2 AI 不擅长哪类 Bug

AI 不适合单独决定：

状态机最终业务口径
是否允许手动推进
是否新增 P2 功能
是否扩大接口范围

是否修改公共模块
是否直接合并

这些问题需要团队根据需求、演示目标和跨模块影响判断。

7.3 AI 是否出现只修表象

可能出现。

例如：

给 HTTP 回调增加更长超时
对 `smoke test` 跳过失败步骤
在前端隐藏按钮但不改后端
CI 中只加一个 `echo`

这些方案不会解决根因。

因此，Prompt 必须要求：

说明根因
列出证据
给出最小修复
说明跨模块影响
设计回归测试

7.4 什么上下文最有帮助

提供以下内容后，AI 建议会明显变好：

报错
复现步骤
Controller
Service
状态机
数据库表关系
CI 文件
测试脚本
最近修改
预期业务口径
禁止扩大范围

八、本地验证回填区

以下内容必须由团队在正式分支中执行后填写。

Bug	修改 commit	构建 结果	测试结果	回归结果	证据位置	最终结论
Bug 1： 状态推进 冲突	450c5e4	通过	OrderStatusMachineTest 全部断言通过	主流程 回归通过	GitHub Actions CI 绿色	已修复，接口层口径统一
Bug 2： Delivery 锁等待	65a2dca	通过	DeliveryServiceTest 通过	上传交付 端到端验证通过	本地 mvn test 日志	已修复，独立事务提交后再回调
Bug 3： CI/CD 缺口	602e222	通过	CI 流水线最新运行成功	推送后 自动触发构建	GitHub Actions 运行记录	已补全后端编译/测试/打包步骤

九、最终结论

本次实验表明：

AI 对跨文件规则冲突、事务边界和 CI 缺口具有明显的定位加速作用。
但最终修复可信度不来自 AI 自信程度，而来自代码证据、测试结果、数据库状态和 Git diff。

团队最终采用：

AI 提速
人工定边界
状态机保证业务一致性
测试保证可信度
CI 保证持续验证
Git diff 保证合并质量